UPTEC XX XXXXX
Degree project 30 credits
April, 2026

Heterogeneous Multi-Robot Collaboration Using Language Models

Exploring different sizes and architectures

Rohit Joseph Mamutil

Master's Programme in Embedded Systems

Heterogeneous Multi-Robot Collaboration Using Language Models

Rohit Joseph Mamutil

Abstract

Heterogeneous warehouse fleets typically include robots with different roles, such as carrier vehicles and picker agents, that must coordinate task choice, role synchronization, and recharging while sharing shelves, corridors, and charging stations. This creates a discrete multi-agent decision problem in which throughput depends not only on motion planning, but also on when agents should commit to tasks, wait for support, or allocate time to battery recovery.

This thesis evaluates whether off-the-shelf language models (LMs) can support high-level decision-making for heterogeneous multi-robot collaboration in the Battery-TA-RWARE task-assignment warehouse environment. The control loop summarizes the current environment state, queries an LM for a grounded macro-action, and validates the proposed action against environment-defined feasibility constraints before execution. The experiments compare model size, planning architecture, and prompt representation, using delivery throughput, LM call efficiency, and observed failure behaviour as the main evaluation criteria.

The completed experiments show that, in the earlier shelf-delivery controller, shared-context planning performed better than centralized planning and natural-language prompting performed better than JavaScript Object Notation (JSON) prompting. In the later call-aware shared-context controller, a mid-sized local model (llama3.2:3b) gave the strongest practical trade-off among the completed natural-language runs, while the completed JSON runs remained substantially weaker than their natural-language counterparts. Across the evaluated settings, productive behaviour depended on rule-based safeguards such as action validation and candidate shaping, indicating that LMs were most useful as constrained high-level decision components rather than as unconstrained controllers.

Faculty of Science and Technology

Uppsala University, Place of publication Uppsala

Supervisor: Xuezhi Niu Subject reader: Didem Gürdür Broo

Examiner: Ramanathan Thinniyam Srinivasan

Sammanfattning

I heterogena robotflottor i lager används ofta olika robottyper (t.ex. bärande automated guided vehicles (AGVs) och lastningsagenter) som måste samordna sina handlingar för att uppnå hög genomströmning samtidigt som säkerhetskrav, såsom batterinivåer, uppfylls. Att konstruera styrsystem för sådana multi-robot systems (MRSs) är svårt på grund av stora diskreta handlingsrymder, glesa belöningssignaler och behovet av robust koordination.

Detta examensarbete undersöker om färdigtränade language models (LMs) kan användas som beslutsfattare på hög nivå för samarbete mellan heterogena robotar i en lager-miljö med uppgiftsallokering och batteridynamik (Battery-TA-RWARE). Metoden sammanfattar det multi-agentiska tillståndet i naturligt språk, låter en LM välja en handling per agent och validerar sedan handlingarna mot miljöns tillåtna handlingsmasker.

Arbetet levererar en första utvärderingspipeline för LM-styrning av multi-agentisk lagerlogistik samt preliminära resultat som visar att en enkel språkbaserad LM-policy presterar avsevärt sämre än en domänheuristik, vilket motiverar fortsatt arbete med bättre struktur, grounding och koordination.

Acknowledgement I would like to thank my supervisor, Xuezhi Niu, for continuous guidance and constructive feedback throughout this thesis work. I also thank my reviewer, Didem Gürdür Broo, and my examiner, Ramanathan Thinniyam Srinivasan, for their input on the direction and quality of the work. Finally, I am grateful to the developers and maintainers of the open-source environments and tools used in this project.

Contents

Acronyms **vi**

1 Introduction **1**

1.1 Research Questions 3

1.2 Contributions 4

1.3 Report Outline 5

2 Background and Related Work **5**

2.1 Multi-Robot Systems and Coordination 5

2.2 Warehouse Task Assignment Environments 6

2.3 Language-Model-Based Planning and Grounding 7

2.4 Why Study Model Size and Prompt Design? 7

3 System and Method **8**

3.1 Overview 8

3.2 Environment: Battery-TA-RWARE 8

3.3 Language-Model Control Loop 9

3.4 Experiment Families and Objectives 11

3.5 Planning Architectures 12

3.6 Prompt Structures 13

3.7 Comparison Axes 13

3.8 Evaluation Metrics 14

3.9 Scenario and Model Matrix 14

3.10 Experimental Protocol (Current Stage) 15

3.11 Delimitations 16

3.12 Ethical and Sustainability Considerations 16

4 Preliminary Results and Discussion **17**

4.1 Result Scope and Objective Progression 17

4.2 Objective 1: Shelf-Delivery Experiments 17

4.2.1 Planning architecture 18

4.2.2 Prompt representation 18

4.3 Objective 2: Call-Aware Shelf-Delivery Experiments 19

4.3.1 Prompt representation 19

4.3.2 Adaptation to the changed objective 20

4.3.3 Model size 20

4.3.4 Call-efficiency outcome 21

4.4 Objective 3: Battery-Aware Objective 22

4.5 System-Level Lessons from the Experiments 22

4.6 Limitations of Interpretation 23

5 Conclusions and Future Work **23**

5.1 Future Work 24

A Appendix **28**

A.1 Reproducibility: Running the Controllers 28

A.2 Prompt Excerpt (Illustrative) 29

A.3 Metric Definitions 29

Acronyms

AAMAS Autonomous Agents and Multiagent Systems. 5

AGV automated guided vehicle. ii, 1, 6, 8, 9, 13, 15

JSON JavaScript Object Notation. i, 3–5, 11, 13–15, 18–24

LM language model. i, ii, 1–3, 5–13, 15, 17, 19–25, 29

MARL multi-agent reinforcement learning. 2, 5, 25

MAS multi-agent system. 5, 6

MRS multi-robot system. ii, 2, 5, 6

PPO proximal policy optimization. 25

TA-RWARE task-assignment robotic warehouse environment. 8

1 Introduction

Accounts of language use and language evolution have often linked language to coordination, cooperation, and shared intentionality in social groups [5, 14, 6]. In these accounts, language is not only a means of describing the world, but also a means of aligning intentions, communicating plans, and organizing joint activity. This perspective motivates the present thesis: if language is closely associated with coordination in human settings, then language-capable models may also be worth studying as decision components in multi-agent systems, provided that the setting is concrete enough to test that idea under operational constraints.

Warehouse automation provides such a setting. In the warehouse domain studied here, heterogeneous robot fleets must coordinate under shared spatial and task constraints: carrier automated guided vehicles (AGVs) transport shelves, picker agents support load and unload operations, and both roles compete for shelves, corridors, and charging stations while still trying to achieve a shared operational objective. The coordination problem is therefore not only where agents should move, but also which shelf should be served next, when agents should wait for support, when a carried shelf should be delivered or returned, and when battery recovery should take priority over immediate task progress.

Path planning alone is not sufficient in this setting. Productive coordination also requires agents to synchronize role-dependent actions, react when progress stalls, and adapt when the global objective changes. For example, an AGV may reach a shelf and still fail to make progress if a picker is not synchronized in time; repeated lack of position change can indicate interaction conflicts in the environment and motivate trying a different action; and a controller may need to shift behavior when the objective changes from maximizing deliveries alone to also reducing unnecessary language model (LM) calls or respecting battery-related constraints. The thesis therefore studies the higher-level decision layer above low-level motion: selecting grounded macro actions that determine what each agent should do next under the current task, coordination, and battery conditions.

To evaluate this decision layer in a controlled way, the thesis uses Battery-TA-RWARE, an adaptation of the Task-Assignment Robotic Warehouse environment [9]. Battery-TA-RWARE is a simplified but structured testbed that retains the key coordination features of the warehouse problem: heterogeneous agent roles, requested shelves, shared corridors, charging stations, and explicit load/unload dependencies between AGVs and pickers. At the same time, it abstracts away continuous low-level control by handling motion internally, making it suitable for studying whether better high-level decisions improve coordination under grounded discrete actions.

Multi-robot systems (MRSs) provide useful background for this problem. Early surveys emphasized group architecture, resource conflicts, and learning [4], while later overviews highlighted communication, task allocation/control, and motion coordination as recurring themes [2]. In the warehouse setting of this thesis, these themes translate into concrete questions about assigning requested shelves, synchronizing heterogeneous roles at shelf locations, handling conflicts in shared corridors, and respecting energy-related constraints without stalling delivery progress.

Many approaches address such coordination problems using optimization, hand-engineered heuristics, or multi-agent reinforcement learning (MARL). Optimization and heuristic methods can be effective, but they often require substantial domain-specific design and retuning when the operating conditions or objective formulation change. MARL can learn coordinated policies, but training remains expensive, strongly dependent on reward design and experimental protocol, and often difficult to adapt when the environment, agent mix, or optimization target changes. Benchmarking studies further show that conclusions in multi-agent learning are highly task- and metric-dependent, which makes careful experimental framing important [13]. Recent work in heterogeneous multi-robot settings has also explored bio-inspired *symbiosis* framings and reward shaping strategies to stabilize cooperation under centralized training and decentralized execution [11, 12].

LMs offer a complementary alternative in this space. In this thesis, LMs refer to prompted pre-trained text-generation models that are used as high-level decision components, rather than as low-level controllers or task-specific policies trained directly inside the warehouse environment. Instead of learning a task-specific policy from scratch, an LM can be prompted with a structured summary of the current environment state and asked to choose a grounded macro action, meaning a validated high-level target choice such as a shelf, goal station, or charging station. In this thesis, a high-level decision means selecting a discrete environment action identifier, such as moving toward a shelf, a goal station, or a charging station, rather than generating free-form robot commands. Recent robotics work has used LMs together with explicit grounding and feasibility checks so that language reasoning remains tied to validated actions instead of unconstrained text outputs [1]. This makes LMs a plausible object of study for warehouse coordination, but it also raises a concrete question: can they support productive heterogeneous coordination under discrete action constraints, limited context, and changing objectives?

The thesis studies this question along three main design axes. The first is planning architecture: whether one prompt should produce decisions for all agents that currently need planning, or whether agents should be planned one at a time while still receiving shared global context. The second is prompt representation: whether the same decision-relevant state should be expressed as compact natural language or as a more explicitly

structured JavaScript Object Notation (JSON) prompt. The third is model size, since the experiments compare smaller and larger prompted language models under the same warehouse setting.

This thesis studies LM-driven decision-making in that grounded warehouse setting. The evaluated control loop summarizes the current state into a compact prompt, queries an LM either jointly or per agent depending on architecture, parses the returned action identifier, and validates it against the environment's current feasible-action mask, that is, the set of actions currently allowed for that agent, before execution. Depending on the runner, the controller may both restrict the candidate actions shown in the prompt and validate the selected action after generation, so feasibility is enforced by both candidate shaping and post-selection checking rather than by a free-form output alone. The controller operates in discrete simulation time: the system state is evaluated at every environment step, but LM queries are issued only when an agent requires a new high-level decision. The controller can also use the observed consequence of executed actions after a simulation step, for example whether progress occurred, whether an agent remained blocked or busy, or whether replanning is needed. This makes the approach neither unconstrained continuous control nor pure event triggering, but a step-synchronous control scheme with conditional replanning. Because the controller depends on repeated model inference, the study evaluates not only delivery throughput but also call efficiency, since unnecessary model queries increase latency and computational cost without improving coordination. The experimental program also asks whether the same control framework remains useful when the objective changes, first from pure shelf-delivery throughput to call-aware delivery, and later toward battery-aware behaviour. In this thesis, observed failure behaviour refers to outcomes such as zero-delivery episodes, long no-delivery tails, invalid decisions, or repeated coordination breakdowns such as waiting loops and unresolved support dependencies. The introduction that follows therefore focuses on which LM-based configurations are most effective for productive coordination in this constrained warehouse setting.

1.1 Research Questions

The thesis is guided by the following main research question:

> **Main research question:** To what extent can prompted LMs support grounded high-level decision-making for heterogeneous multi-robot coordination in a constrained warehouse environment?

To support a systematic investigation of the main research question, the study considers the following sub-questions: In this thesis, *centralized planning* means that one prompt

receives the shared state and returns decisions for all agents that currently need planning, whereas *shared-context per-agent planning* means that agents are planned one at a time, but each prompt still contains a shared summary of the global warehouse state.

1. **RQ1 (Model size):** How does language-model size affect coordination performance in the warehouse setting, measured by delivery throughput, language-model calls per delivery, and observed failure behaviour across scenarios?
2. **RQ2 (Planning architecture):** How do centralized and shared-context per-agent planning architectures differ in coordination performance, measured by delivery throughput, robustness across scenarios with different coordination pressure, and the frequency of non-productive runs or coordination failures?
3. **RQ3 (Prompt representation):** How do natural-language and JSON prompt representations differ in coordination performance, measured by delivery throughput, call efficiency, and the consistency of valid, productive decisions?

In this thesis, these questions are evaluated primarily through delivery throughput, language-model call efficiency, and observed failure behaviour. Depending on the experiment family, the analysis also considers robustness across scenarios, the prevalence of zero-delivery or long no-delivery runs, and whether productive coordination is maintained when the objective changes.

1.2 Contributions

The current thesis work contributes:

- A grounded evaluation setting for studying language-model-based high-level control in heterogeneous warehouse coordination, where proposed decisions are translated into discrete environment actions and checked against explicit feasibility constraints rather than executed as free-form outputs.
- Comparative evidence on three design variables relevant to the research questions: model size, planning architecture, and prompt representation. In particular, the experiments provide a basis for assessing how these choices affect delivery throughput, call efficiency, robustness across scenarios, and the prevalence of non-productive behaviour.
- Empirical insights into the conditions under which the approach remains productive. The current results indicate that shared-context per-agent planning is more

promising than centralized planning in the evaluated shelf-delivery setting, that natural-language prompting is more effective than JSON prompting in the completed comparisons, and that language-model-based control depends strongly on grounding and rule-based safeguards to avoid systematic failure modes such as no-delivery tails, stale waiting loops, and other coordination breakdowns.

1.3 Report Outline

Section 2 provides background on multi-robot coordination, warehouse task assignment, and LM-based planning. Section 3 describes the environment, the proposed LM control loop, the experimental design, and the main delimitations of the study. Section 4 presents the current experimental results together with their interpretation. Finally, Section 5 concludes and outlines future work.

2 Background and Related Work

2.1 Multi-Robot Systems and Coordination

MRSs study how multiple embodied agents share resources, divide work, communicate, and avoid conflicts while pursuing a common objective. The term multi-agent system (MAS) is broader and can include purely software agents, whereas MRS typically emphasizes physical robots operating under motion, sensing, and actuation constraints. Classical overviews of cooperative robotics highlight recurring concerns such as group architectures, resource conflicts, and learning [4]. Survey treatments vary in terminology and scope (e.g., differences in how MAS and MRS are framed, and how communication and coordination taxonomies are defined) [16, 2]. For broader multi-agent coordination perspectives, conference proceedings such as Autonomous Agents and Multiagent Systems (AAMAS) provide additional representative examples, including coordinated MARL methods that explicitly trade off coordination benefits against communication bandwidth [18]. In heterogeneous teams, these concerns intensify because roles and capabilities differ, and success often requires explicit synchronization (e.g., meeting at the same physical location at the same time).

In this thesis, *collaboration* refers to interdependent task execution among heterogeneous roles, where agents must coordinate to complete an operation (e.g., a load/unload interaction). *Coordination* refers to mechanisms that prevent conflicts and deadlocks (e.g., collision avoidance and congestion handling), and *cooperation* refers to pursu-

ing a shared objective even when agents may act more independently. These terms are not used consistently across communities; some MAS discussions historically emphasize distributed stationary nodes, whereas modern multi-agent learning often includes mobile, embodied agents. To reduce ambiguity, the thesis primarily uses MRS when referring to embodied robots and uses MAS to denote the broader class of multi-agent systems.

Coordination strategies can be described as *static* (offline, convention-based) or *dynamic* (online, reactive), depending on whether coordination rules are established prior to execution or synthesized during execution from sensed/communicated information [16]. Communication can likewise be categorized as *implicit* (information inferred via sensing or environment traces) versus *explicit* (direct message passing), trading off scalability against accuracy and bandwidth [16].

Planning is a central lens for collaboration: multi-robot planning is often discussed as task planning (who does what) and motion planning (how to move), with optimal formulations typically being NP-hard and thus motivating approximate methods and heuristics [16]. In particular, multi-robot task planning includes task decomposition (splitting a mission into subtasks) and task allocation (assigning subtasks to agents), with much of the literature focusing on allocation due to tractability and benchmarking convenience [16]. This observation aligns with the thesis hypothesis: LMs may be useful as a structured decision module for decomposition and allocation when paired with explicit grounding, feasibility constraints, and environment feedback.

2.2 Warehouse Task Assignment Environments

Warehouse domains provide a structured but challenging testbed for multi-agent decision-making. Tasks are naturally decomposed into repeated cycles: select a requested shelf, bring it to a delivery station, return it to storage, and repeat. When multiple agents operate in the same grid, the action space and coordination requirements scale quickly with environment size and fleet size.

This thesis uses Battery-TA-RWARE, an adaptation of the Task-Assignment Robotic Warehouse environment introduced in [9]. The environment models two heterogeneous agent types:

- **AGVs:** carrier agents that transport shelves between storage locations and goal stations.
- **Pickers:** support agents that must meet AGVs at shelf locations for loading and unloading.

Battery-TA-RWARE extends the task-assignment setting with battery depletion and charging stations, forcing agents to balance throughput with energy safety. Low-level movement is handled by environment dynamics (including A* path planning), while the controller selects discrete targets representing shelves, goals, or charging stations.

2.3 Language-Model-Based Planning and Grounding

LMs can produce coherent natural-language plans, but robot control requires converting language into *grounded*, *feasible* actions. One influential pattern is to constrain the LM to select among validated, environment-defined options. For example, SayCan combines language-level reasoning with an affordance model to filter actions that are executable in the current state [1]. Other work frames LMs as planners that can propose action sequences given a symbolic interface to the environment, often coupled with explicit checking and replanning [7]. In language-only settings, prompting patterns such as interleaving reasoning and actions have been studied to improve decision quality under tool or environment feedback [17]. Recent multi-robot systems work explores *multi-agent* prompting where each robot maintains a separate dialog context and coordination emerges through structured communication plus environment feedback (e.g., collision checking and in-context plan revision) [10]. Recent work argues that closed-loop state feedback and a separation between high-level planning and low-level control can reduce hallucinated actions and improve long-horizon goal satisfaction in robot task planning with LMs [3]. In multi-robot task planning contexts, prior thesis work also emphasizes the need for a symbolic abstraction layer that translates sensor/state representations into language-level predicates that LMs can reliably consume [8].

These ideas motivate the design in this thesis: use a prompt that provides a compact but explicit mapping from environment state to action identifiers, require outputs to be parseable into discrete actions, and validate them against the environment's current action constraints.

2.4 Why Study Model Size and Prompt Design?

In applied robotics and embedded contexts, compute and latency budgets may favour smaller models. However, smaller models may require more structured prompting, tighter candidate sets, and stronger validation to reach acceptable performance. Recent robotics work explicitly motivates prompt design as a lever for compensating limited model capacity and improving decision quality in constrained deployments [15, 10]. By evaluating multiple model sizes and prompt variants under the same environment and

metrics, the thesis aims to clarify which components (model capacity, prompt structure, validation) most strongly affect feasibility, coordination quality, and throughput.

3 System and Method

3.1 Overview

The thesis uses an empirical, experimental methodology:

1. Implement an LM-driven control loop for a simulated heterogeneous warehouse environment.
2. Define the main comparison axes and evaluation metrics.
3. Run controlled experiments (fixed environment ids and seeds) across multiple LM models, planning architectures, and prompt variants.
4. Analyze throughput, safety, and coordination outcomes as well as qualitative failure modes.

3.2 Environment: Battery-TA-RWARE

Battery-TA-RWARE is a discrete-time, grid-based warehouse simulation derived from task-assignment robotic warehouse environment (TA-RWARE) [9]. At any time, a set of shelves is requested. Carrier AGVs must transport requested shelves to goal locations and then return them to storage. Picker agents are required to meet AGVs at shelf locations for load and unload interactions, creating explicit heterogeneous coordination constraints. Agents additionally have a battery state that decreases with movement and interactions; charging stations are available at fixed locations.

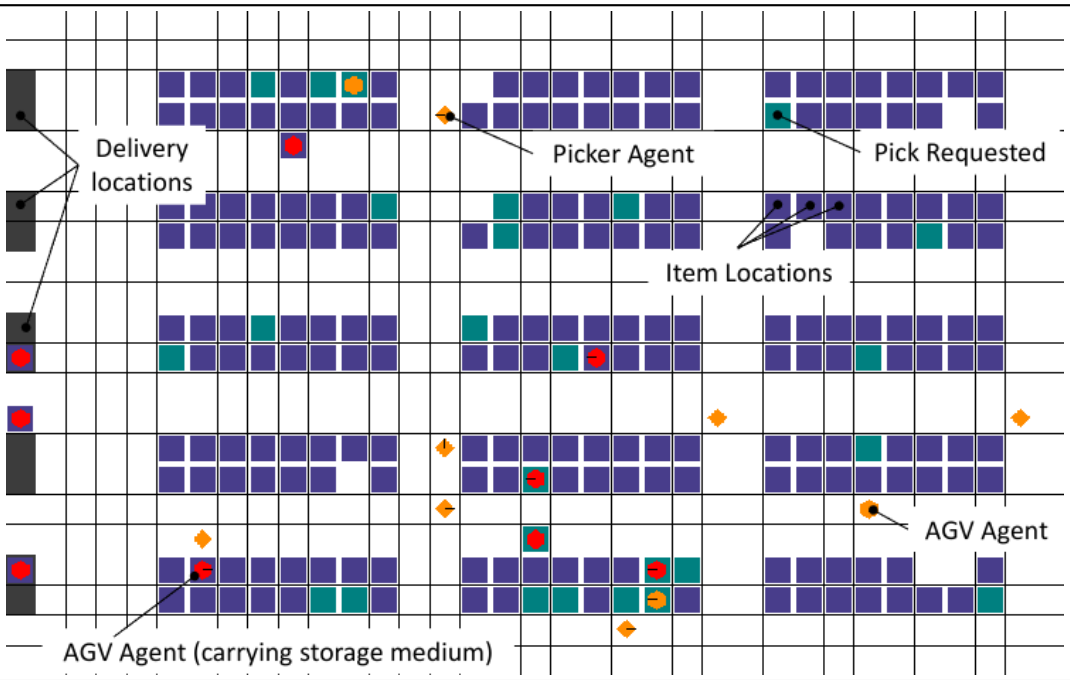

Figure 1 Illustration of the task-assignment warehouse setting used in this thesis. Two heterogeneous roles must coordinate: carrier AGVs transport shelves, while picker agents support loading and unloading at shelf locations. Battery dynamics add a safety constraint that can force agents to recharge.

The environment exposes a discrete action space where each action corresponds to selecting a target location:

- **Shelf actions:** choose a shelf location (e.g., to pick up or return a shelf).
- **Goal actions:** choose a delivery station (primarily relevant when an AGV carries a requested shelf).
- **Charging actions:** choose a charging station to recover battery.

Low-level motion toward the selected target is handled internally by the environment (A* path planning and collision resolution), while the controller is evaluated on the quality of these high-level decisions.

3.3 Language-Model Control Loop

The LM policy is implemented as a text-in / discrete-action-out controller:

1. **State summarization:** extract salient features of the global state (requested shelves and their action ids, per-agent position/target/battery/carrying status, charging occupancy).
2. **Candidate action set:** compute a valid-action mask from the environment; optionally restrict the candidate list shown to the LM to reduce prompt size.
3. **Prompting:** generate a per-agent prompt describing the agent role and the current state, including explicit mappings from requested objects to action ids.
4. **Querying:** send the prompt to a local inference server (Ollama) and obtain a text response.
5. **Parsing:** parse an integer action id (and optionally an action hold duration) from the response.
6. **Validation:** validate the parsed action id against the current valid-action mask; invalid outputs are discarded and trigger replanning.
7. **Execution:** apply the validated joint action in the environment for one step (or hold the action for multiple steps, depending on configuration).

The controller is best described as a discrete-time, step-synchronous supervisory loop with conditional replanning. The environment advances one simulation step at a time, and at each step the controller reevaluates agent state and action validity. However, this does not imply that every agent is queried at every step. Instead, the experiments use two related control schemes.

- **Step-synchronous replanning with environment-owned continuation:** agents that are currently available for planning may receive a new high-level action, while busy agents are not replanned and continue their current task until the environment releases them.
- **Step-synchronous replanning with action persistence and conditional override:** accepted actions may remain active for multiple future steps, so agents are not re-queried while the persisted action remains valid. Replanning is triggered when a persisted action expires, becomes invalid, or when the controller decides intervention is needed, including selected busy-agent cases.

This design sits between two extremes. A fully scheduled scheme that queries every agent at every step would generate many unnecessary LM calls, while a purely event-triggered scheme is less natural in this setting because useful triggers such as blocked

progress, invalid persisted actions, expiring commitments, or changing support needs are only known after the system state is checked. The adopted design therefore combines regular per-step observability with selective replanning. This is also consistent with practical supervisory control in warehouse systems, where the fleet is typically checked at regular intervals, ongoing tasks continue by default, and new task-level decisions are issued only when the current situation justifies intervention.

Two design choices are central for feasibility:

- **Grounding via action ids:** the LM does not output coordinates or free-form text commands; instead it selects from discrete action ids that are already defined by the environment.
- **Hard validation:** action masks enforce constraints such as invalid goals for picker agents and busy-state restrictions. Invalid LM outputs are never executed.

3.4 Experiment Families and Objectives

The implemented experiments are grouped into three objective families.

Objective 1: maximize total shelf deliveries. The first family focuses on raw throughput. These experiments are stored under results/shelf and were designed to answer two questions: whether a centralized or shared-context planning architecture is more effective, and whether natural-language or JSON-style prompting is the more suitable representation for that objective. The family was implemented in two architectures:

- **Centralized planner:** scripts/run_centralized_llm_experiments.py. One LM prompt receives the global state and returns a joint plan for all non-busy agents.
- **Shared-context decentralized planner:** scripts/run_shared_context_llm_expt.py. Each agent is planned separately, but each prompt still contains a shared global summary of requests, charging occupancy, and waiting support relations.

For both architectures, two prompt styles were evaluated:

- **Natural-language form:** the prompt and response use a compact textual format.
- **JSON form:** the prompt and response use a more structured object-and-array representation.

Objective 2: optimize LM calls while maintaining delivery throughput. The second family is implemented in scripts/run_obj2_shared_context_llm.py and stored under results/calls. This family keeps the shared-context, per-agent architecture, but changes the objective from delivery-only to a joint objective: increase shelf deliveries while reducing unnecessary LM queries. In practice, the runner uses action holding and conditional replanning so that the model is not queried for every agent at every environment step. This family was used primarily to study the practical trade-off between throughput, model size, and call efficiency, and more broadly to test whether the LM-based control framework remained useful when the objective changed.

Objective 3: battery-aware objective. A third family was planned for explicitly battery-oriented evaluation and corresponding result storage under results/battery. At the time of writing, this experimental branch had not yet produced completed result files, so it is described as planned work rather than evaluated work.

3.5 Planning Architectures

The experiments compare two distinct ways of integrating the LM into the decision loop.

Centralized planning. In the centralized architecture, one LM call plans all agents that currently need a decision. This design exposes the full global state and all relevant candidate actions in one prompt. Its intended advantage is explicit joint coordination. Its main risk is prompt overload: as the number of agents and candidate actions increases, the prompt becomes large and more difficult to parse reliably.

Shared-context per-agent planning. In the shared-context architecture, the LM plans one agent at a time. Each prompt still includes global information, but the final decision concerns only the current agent. This reduces the output burden and simplifies parsing, at the cost of requiring coordination to emerge from repeated local decisions rather than from one explicit joint plan. In the thesis evaluation, this architecture is the main vehicle for comparing prompt forms and model sizes under a common decision structure.

The run_obj2_shared_context_llm.py variant extends the shared-context architecture with explicit action-hold durations and logging for call-efficiency analysis. It therefore remains decentralized in prompt structure while adding stronger runner-side control over

when replanning is allowed.

3.6 Prompt Structures

The prompt comparison in this thesis is not limited to natural language versus JSON. The prompt design varies along two axes:

- **Planning scope:** centralized joint planning versus per-agent planning with shared context.
- **Output structure:** natural-language responses versus JSON responses.

Across the different scripts, the prompts evolved to include:

- explicit requested-shelf to action-id mappings,
- role-specific rules for AGVs and pickers,
- support relations such as waiting-for-picker context,
- battery-state abstractions,
- candidate action filtering or prioritization,
- and, in Objective 2, an explicit Steps field intended to reduce unnecessary LM calls.

The purpose of this progression was experimental rather than aesthetic: to determine which combination of planning scope and prompt structure yields valid, productive macro actions under the constraints of the warehouse task. In particular, the prompt comparison was intended to separate two questions: whether planning should be centralized or per-agent, and whether the same decision content is handled better in natural-language form or in a more structured JSON form.

3.7 Comparison Axes

The current report focuses on internal comparisons within the LM-based control framework rather than on comparison against an external baseline. The main comparison axes are:

- model size,
- planning architecture (centralized versus shared-context per-agent),
- prompt form (natural language versus JSON),
- and objective formulation (shelf-delivery only, call-aware shelf delivery, and planned battery-aware control).

3.8 Evaluation Metrics

The main metric is delivery throughput, expressed as a pick rate (deliveries per hour) derived from the number of delivered shelves and the simulated episode duration. Additional metrics are collected to capture coordination and safety:

- total shelf deliveries,
- number of clashes and deadlocks/stuck events,
- distance travelled by each agent type,
- episode return (environment reward),
- episode length and inference speed/latency (when measured).

3.9 Scenario and Model Matrix

The same scenario catalogue is reused across the experiment families:

- tiny_balanced_1v1,
- small_balanced_2v2,
- medium_balanced_4v4,
- small_agv_heavy_4v2,
- medium_agv_heavy_6v2,
- small_picker_heavy_2v4.

These scenarios were selected to probe different coordination pressures: simple balanced collaboration, increased concurrency, larger warehouses, AGV-heavy support bottlenecks, and picker-heavy underutilization.

The model set spans smaller and larger local models:

- llama3.2:1b,
- llama3.2:3b,
- mistral:7b,
- gemma3:12b,
- phi4,
- qwen2.5:14b.

3.10 Experimental Protocol (Current Stage)

To support reproducibility during prompt iteration, the reported experiment sessions were run with fixed environment ids and a fixed seed. This choice makes behavioural debugging easier, but it also limits statistical confidence. The current thesis therefore treats the results as experimental evidence about architecture and prompt behaviour, not as final performance claims.

The progression of the experimental campaign was:

1. Objective 1 shelf-delivery experiments, comparing centralized and shared-context planning and comparing natural-language and JSON prompt forms,
2. Objective 2 call-aware shelf-delivery experiments, using a later shared-context controller with stronger prompt structure and safeguards,
3. Objective 3 battery-aware experiments, planned but not yet completed.

One broader aim of this progression was to test whether the same general LM-based control framework could adapt as the stated objective changed. Section 4 therefore first analyzes Objective 1 as the shelf-delivery baseline, then Objective 2 as a changed objective using a later and more structured shared-context controller, and finally notes Objective 3 as planned future evaluation. In the current simulation setup, a run can achieve at most 20 deliveries, and the results chapter uses that ceiling to contextualize reported scores.

3.11 Delimitations

To keep the study focused and interpretable at the current stage, the following delimitations apply:

- The work is evaluated in a simulated warehouse environment rather than on physical robots.
- The controller is responsible only for high-level target selection; low-level navigation and collision handling are performed by the environment.
- The evaluated language models are prompted general-purpose models accessed through a local inference server and are not fine-tuned for this task.
- The reported experiments use fixed seeds and a limited number of completed sessions, so the results should be interpreted as exploratory rather than statistically conclusive.
- The battery-aware objective family had not yet produced completed aggregate results at the time of writing.

3.12 Ethical and Sustainability Considerations

Although the present work is simulation-based, it raises practical issues that remain relevant for real multi-robot deployments. The primary ethical concern is safety and reliability: generated decisions may be infeasible, unstable, or unproductive if they are not constrained by system-side checks. For that reason, the studied control loop does not execute free-form outputs directly. It uses grounded action identifiers, candidate shaping, explicit feasibility validation, and fallback behavior so that unsafe or clearly invalid actions are filtered before execution. The logging of prompts, parsed outputs, executed actions, and observed step consequences also improves traceability, which is important when analyzing failures such as long no-delivery tails, stale waiting loops, or battery-related breakdowns.

From a sustainability perspective, the main trade-off is between model capability and computational cost. Larger language models generally require more inference time and compute, while smaller models may require stronger prompt structure and validation to remain effective. This thesis therefore treats call efficiency, unnecessary replanning, and coordination overhead as practically relevant concerns rather than only implementation details. Even with safeguards, the current study does not eliminate all risks: fixed-seed

evaluation, limited scenario coverage, and the absence of physical deployment mean that conclusions about safety, efficiency, and generalization remain provisional.

4 Preliminary Results and Discussion

4.1 Result Scope and Objective Progression

This chapter analyzes the completed experiment families stored under results/shelf and results/calls. The experimental program was organized around three objectives. Objective 1 focused on maximizing total shelf deliveries. Objective 2 retained shelf delivery as the task outcome, but changed the optimization target to improving delivery while reducing unnecessary LM calls. Objective 3 was planned as an explicitly battery-aware objective, but that branch had not yet produced completed aggregate results at the time of writing.

One broader goal of the thesis was to test whether an LM-based control framework could remain useful when the objective changed. In other words, the question was not only which configuration performed best, but also whether the same overall control paradigm could move from pure throughput optimization to a joint throughput-and-call-efficiency objective, and later toward a battery-oriented objective.

For interpreting the reported numbers, one additional fact is important: in the current simulation setup, 20 deliveries is the maximum achievable total for a run. Reported delivery counts should therefore be read relative to this ceiling. A run with only a few deliveries is not merely weak in absolute terms; it is far below what the environment allows when coordination succeeds.

Objective 2 should also not be read as a direct continuation of Objective 1 under an unchanged controller. It used a later shared-context runner with stronger prompt design, action holding, and additional safeguard logic. The Objective 2 results therefore reflect both a changed objective and a more mature controller.

4.2 Objective 1: Shelf-Delivery Experiments

Objective 1 provides the main basis for comparing planning architecture and prompt form under the simplest thesis objective: maximize total shelf deliveries. These experiments are stored under results/shelf and reflect an earlier stage of the system, before the later call-aware shared-context runner was introduced.

4.2.1 Planning architecture

The architecture comparison in Objective 1 is summarized in Table 1.

Experiment family	Runs	Total deliveries	Average deliveries/run
Centralized + natural	36	15	0.42
Centralized + JSON	36	21	0.58
Shared-context + natural	36	25	0.69
Shared-context + JSON	36	0	0.00

Table 1 Aggregate outcome of the Objective 1 shelf-delivery experiments across models and scenarios.

The overall pattern suggests that shared-context planning was more promising than centralized planning, particularly when paired with natural-language prompting. Shared-context plus natural language produced the highest aggregate delivery count in this family. The likely reason is that per-agent planning reduced the burden of each query while still exposing the global state needed for coordination. In the centralized architecture, one prompt had to describe the full state and return multiple coordinated actions at once, which created a heavier joint decision problem for the tested models.

At the same time, the Objective 1 results should not be overstated. Even the best configuration in this family remained far below the 20-delivery per-run ceiling. This means Objective 1 is more useful as an architecture comparison than as evidence of strong final performance. It suggests that shared-context planning was the more promising direction, but it does not demonstrate that the Objective 1 controller had already become effective in absolute terms.

4.2.2 Prompt representation

Objective 1 is also the main basis for comparing prompt representation under the shelf-only objective. In this family, natural-language prompting performed better than JSON prompting in the shared-context architecture. The recorded shared-context natural-language runs produced 25 deliveries across 36 runs, whereas the recorded shared-context JSON runs produced zero. The centralized comparison was less one-sided, but all centralized variants remained weak overall and still far below the per-run delivery ceiling.

The correct interpretation is cautious. The early JSON prompts were not fully comparable to the natural-language prompts. They used deeper nesting and exposed some

decision-critical state less directly. Later prompt revisions made the JSON form much closer to the natural prompt in information content and ordering. For that reason, the early Objective 1 JSON results should not be taken as proof that structured prompting is inherently unsuitable. They do, however, show that in the earlier shelf-delivery setup, natural language was clearly the more effective prompt form.

Taken together, Objective 1 answers two thesis questions in a preliminary but useful way: shared-context planning looked more promising than centralized planning, and natural-language prompting looked more effective than JSON prompting in the earlier shelf-delivery controller. Both findings should be read as informative rather than definitive because Objective 1 belonged to an earlier stage of the system.

4.3 Objective 2: Call-Aware Shelf-Delivery Experiments

Objective 2 changed the goal from pure delivery throughput to a joint objective: preserve delivery performance while reducing unnecessary LM calls. These experiments are stored under results/calls and use a later shared-context runner with stronger prompt structure, action holding, and additional safeguard logic. For that reason, Objective 2 is the main basis for evaluating model size and for asking whether the LM-based framework remained useful under a changed objective.

Across 36 runs, the completed Objective 2 natural-language shared-context session produced 303 deliveries, corresponding to an average of 8.42 deliveries per run. This was a substantial improvement over the Objective 1 aggregate results, although that improvement cannot be attributed to the objective change alone because the controller itself also became more mature.

4.3.1 Prompt representation

The completed Objective 2 sessions provide a direct prompt-format comparison within the later shared-context runner family. The natural-language session produced 303 deliveries across 36 runs, corresponding to 8.42 deliveries per run. The completed JSON session produced 53 deliveries across the same number of runs, corresponding to 1.47 deliveries per run. Efficiency measures show the same pattern: the natural-language session required approximately 204.7 LM calls per delivery and 469.5 seconds per delivery, while the JSON session required approximately 1078.4 calls per delivery and 3057.4 seconds per delivery.

The later Objective 2 JSON session was not a complete failure, since it produced non-

zero throughput, mainly through gemma3:12b and phi4. However, it remained substantially weaker than the natural-language version of the same runner family. The natural-language session had 2 zero-delivery runs out of 36, whereas the JSON session had 26 zero-delivery runs out of 36. The completed Objective 2 results therefore support the same overall conclusion as Objective 1: natural-language prompting remained more effective than JSON prompting in the evaluated setup, even after the later prompt and runner refinements.

4.3.2 Adaptation to the changed objective

Objective 2 provides partial evidence that the framework remained useful when the optimization target changed from throughput only to throughput plus call efficiency. The later shared-context controller still produced substantial delivery throughput while explicitly trying to reduce unnecessary calls. In several scenarios, the best-performing model reached or approached the 20-delivery ceiling, which indicates that productive control remained possible under the revised objective.

This result is not a clean transfer study, because Objective 2 was not evaluated under a frozen Objective 1 controller. Runner logic, prompt design, and safeguards changed together with the stated objective. The completed experiments therefore support a narrower conclusion: the evolving LM-based framework remained viable under the changed objective, but the available evidence does not isolate objective adaptation from the effect of the later controller improvements.

4.3.3 Model size

The clearest model-size comparison in the thesis comes from Objective 2 and is summarized in Table 2.

Model	Total deliveries	Average deliveries/scenario	Seconds/delivery
llama3.2:3b	73	12.17	238.5
gemma3:12b	63	10.50	460.2
mistral:7b	59	9.83	530.5
qwen2.5:14b	46	7.67	495.6
llama3.2:1b	36	6.00	371.4
phi4	26	4.33	1091.6

Table 2 Aggregate results for the Objective 2 shared-context natural-language session.

Table 2 suggests that model size mattered, but not in a monotonic way. Larger models did not consistently achieve better throughput, nor did they consistently justify their additional latency. The strongest practical operating point in this session was llama3.2: 3b, which combined the highest total deliveries with the lowest seconds-per-delivery among the tested models. This is notable because it is neither the smallest nor the largest model in the set.

The model-size effect was also scenario-dependent. Some scenarios appear to have rewarded richer context interpretation, while others rewarded faster and more consistent local decisions. This means the thesis does not support a universal best-size claim. It does support the narrower conclusion that larger models were not uniformly better, and that a mid-sized local model offered the strongest practical trade-off in the most complete Objective 2 session.

The completed Objective 2 JSON session also adds an important nuance to the model-size picture. In that session, only gemma3:12b and phi4 produced substantial throughput, with 26 and 25 deliveries respectively, while both llama3.2 variants produced zero deliveries across all six scenarios. This suggests that structured prompting in the later call-aware setup placed a heavier burden on the smaller models than the corresponding natural-language prompts did. Even so, the overall prompt-format effect remained larger than the model-size effect: the best JSON models still remained well below the best natural-language results.

4.3.4 Call-efficiency outcome

Objective 2 should be judged on both throughput and LM usage. Raw calls-per-delivery were approximately 146.2 for phi4, 158.3 for gemma3:12b, 167.0 for qwen2.5:14b, 167.7 for llama3.2:3b, 287.0 for mistral:7b, and 316.3 for llama3.2:1b. This metric should still be interpreted with care, since a model can appear efficient simply by doing too little. For that reason, the most convincing result is not the lowest raw call ratio, but the balance between delivery throughput and call usage. By that criterion, llama3.2:3b again appears to be the most practical choice in the completed Objective 2 session.

Objective 2 therefore appears to have made meaningful progress toward the call-aware goal, but only partially. Action holding and reuse reduced some unnecessary queries, yet many query resolutions still arose from runner-side intervention rather than from stable long-horizon plans. The result is best described as a workable but still imperfect call-aware controller rather than a fully solved low-query policy.

4.4 Objective 3: Battery-Aware Objective

Objective 3 was planned to test whether the same general framework could adapt again to a further changed objective centered on battery behaviour. The intended result family is stored under results/battery. At the time of writing, however, this branch had not yet produced completed aggregate results. For that reason, Objective 3 remains future work rather than an analyzable result set.

This means the thesis can only partially answer the broader adaptation question. There is evidence that the framework remained useful when moving from Objective 1 to Objective 2, but there is not yet completed evidence for adaptation to an explicitly battery-oriented objective.

4.5 System-Level Lessons from the Experiments

The strongest overall lesson from Objectives 1 and 2 is that LMs were most useful when bounded by rule-based logic rather than treated as unconstrained controllers. The experiments do not provide positive evidence that giving full responsibility to the LM improved outcomes. Instead, stronger behaviour appeared only when the model operated inside a grounded and safeguarded decision loop with candidate shaping, action validation, and conflict-handling logic.

Several observations support this interpretation. First, Objective 1 remained weak across all architecture and prompt combinations, with most configurations finishing far below the 20-delivery ceiling. Second, the stronger Objective 2 results appeared only in a later shared-context controller that relied on additional structure such as action holding and stronger control logic around when and how decisions were reused. Third, the JSON experiments showed that valid syntax alone was not sufficient: parseable outputs still often led to poor decisions.

The failure patterns point in the same direction. In the Objective 2 natural-language shared-context session, 32 of 36 runs ended with an exact 500-step no-delivery tail. These poor runs often stopped far below the 20-delivery ceiling and were usually not associated with strong explicit conflict or stuck signals. They were more consistent with macro-level coordination failures such as stale waiting loops, support-allocation problems, or charging oscillation. This indicates that higher-level coordination did not become reliable simply because an LM was placed in the loop.

The architectural lesson is therefore narrow but important. In the present system, the LM contributed most when it selected among already grounded and validated macro

actions. Correctness-critical functions such as action validity, exclusivity, and conflict handling remained better suited to explicit rule-based logic. The experiments therefore support a constrained view of LM control rather than a fully free-form one.

4.6 Limitations of Interpretation

Several limitations should be kept in mind when interpreting these results.

- The reported sessions use fixed seeds and should therefore be treated as behavioural evidence rather than as statistically robust rankings.
- The experiment families were not clean ablations. Prompt design, runner logic, and some environment-side behaviour evolved across the objectives, which limits causal attribution.
- Objective 2 should not be read as a pure apples-to-apples continuation of Objective 1, since both the objective and the controller changed.
- Objective 3 had not yet produced completed results, so battery-aware adaptation remains open.
- Some outcomes were scenario-specific and may not persist under broader reruns.

Taken together, the results support four cautious conclusions. Objective 1 established that shared-context planning was more promising than centralized planning and that natural-language prompting was more effective than JSON prompting in the earlier shelf-delivery controller. Objective 2 showed that the framework could remain useful under a changed, call-aware objective, especially in a more structured shared-context setup. Within that stronger family, a mid-sized model offered the best practical trade-off. Most importantly, the experiments suggest that LM control was most viable when bounded by rule-based safeguards rather than treated as a replacement for them.

5 Conclusions and Future Work

This thesis explores LM-driven decision-making for heterogeneous multi-robot collaboration in a task-assignment warehouse setting with battery constraints. The work implemented an end-to-end control loop that (i) grounds decisions to discrete environment action identifiers, (ii) queries an LM using centralized or per-agent shared-context prompts, and (iii) enforces feasibility through action-mask validation.

The results should be interpreted relative to a 20-delivery per-run ceiling in the current simulation setup. Against that ceiling, Objective 1 established the initial shelf-delivery comparison between architectures and prompt forms. In that earlier controller family, shared-context planning was more promising than centralized planning, and natural-language prompting was more effective than JSON prompting. These results were informative, but they remained far below the achievable ceiling and therefore served more as an indication of architectural direction than as evidence of strong final performance.

Objective 2 then changed the optimization target from pure shelf delivery to call-aware shelf delivery while also using a later, safer shared-context runner. The completed Objective 2 sessions show that the framework remained usable under the changed objective, especially in the more structured shared-context setup. Within that family, model size mattered, but not monotonically: larger models were not uniformly better, and the strongest practical trade-off in the natural-language session came from a mid-sized model, `llama3.2:3b`, rather than from the largest available model. The completed Objective 2 JSON session produced non-zero throughput, but it remained substantially weaker than natural language in both throughput and call efficiency.

Objective 3, the battery-aware objective, remains open because completed aggregate result files were not yet available. This means the thesis only partially answers the broader question of whether the framework adapts smoothly as the objective changes. The evidence supports viability across the shift from Objective 1 to Objective 2, but not yet for a dedicated battery-centered objective.

The strongest overall lesson is not that the LM replaced controller logic, but that it contributed best as a constrained macro-action selector inside a safeguarded system. Many poor runs ended in long non-productive tails, often far below the 20-delivery ceiling and usually without clear low-level collision deadlocks. This suggests that higher-level coordination did not become reliable simply by inserting an LM into the loop. Action validation, candidate shaping, conflict handling, and other rule-based logic remained necessary to keep the system stable and productive.

The overall evidence in this thesis therefore supports a constrained view of LM control. In the evaluated warehouse setting, LMs were most useful inside a strongly structured decision loop, while correctness-critical behaviour remained better handled by explicit rule-based logic.

5.1 Future Work

- **Structured outputs and tighter parsing:** require the model to output a strict schema (e.g., {`action_id, hold_steps, rationale`}) and reject non-

conforming responses to reduce ambiguity.

- **Centralized versus decentralized planning:** compare per-agent prompts (current approach) with a centralized planner that outputs a joint action vector, including explicit coordination constraints.
- **Candidate-set design:** learn or compute compact candidate action sets (e.g., nearest requested shelves, relevant support targets for pickers) while guaranteeing that necessary actions remain available.
- **Multi-seed and scaling studies:** extend evaluation to many random seeds and larger layouts (more shelves, more agents) to quantify robustness and scalability.
- **MARL baselines:** train/evaluate proximal policy optimization (PPO) or other MARL methods on Battery-TA-RWARE and compare to LM in throughput, generalization, and compute cost.
- **Cost-aware deployment:** measure inference latency, token usage, and energy cost across model sizes to assess feasibility for embedded or real-time settings.

References

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[2] T. Arai, E. Pagello, and L. E. Parker, "Editorial: Advances in multi-robot systems," *IEEE Transactions on Robotics and Automation*, vol. 18, no. 5, pp. 655–661, 2002.

[3] V. Bhat, A. U. Kaypak, P. Krishnamurthy, R. Karri, and F. Khorrami, "Grounding large language models for robot task planning using closed-loop state feedback," 2025, received: 27 April 2025; revised: 10 October 2025; accepted: 17 October 2025.

[4] Y. U. Cao, A. S. Fukunaga, and A. B. Kahng, "Cooperative mobile robotics: Antecedents and directions," *Autonomous Robots*, vol. 4, no. 1, pp. 7–27, 1997.

[5] H. H. Clark, *Using Language*. Cambridge, UK: Cambridge University Press, 1996.

[6] R. Dunbar, *Grooming, Gossip, and the Evolution of Language*. Cambridge, MA, USA: Harvard University Press, 1996.

[7] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," *arXiv preprint arXiv:2201.07207*, 2022.

[8] Y. Huang, "Collaborative multi-robot task planning using large language models," Master's thesis, Uppsala University, 2025, accessed 2026-02-26. [Online]. Available: https://uu.diva-portal.org/smash/get/diva2:2023514/FULLTEXT01.pdf

[9] A. Krnjaic, R. D. Steleac, J. D. Thomas, G. Papoudakis, L. Schäfer, A. Wing Keung To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Börsting, and S. V. Albrecht, "Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers," 2022.

[10] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics*

and Automation (ICRA), 2024, pp. 286–299, accessed 2026-02-26. [Online]. Available: https://ieeexplore.ieee.org/document/10610855

[11] X. Niu, N. C. Barajas, and D. G. Broo, "Enabling symbiosis in multi-robot systems through multi-agent reinforcement learning," in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–7.

[12] X. Niu and D. G. Broo, "Investigating symbiosis in robotic ecosystems: A case study for multi-robot reinforcement learning reward shaping," in *2025 9th International Conference on Robotics and Automation Sciences (ICRAS)*, 2025, pp. 112–117.

[13] G. Papoudakis, F. Christianos, L. Schäfer, and S. V. Albrecht, "Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks," 2020, accessed 2026-02-26. [Online]. Available: https://arxiv.org/abs/2006.07869

[14] M. Tomasello, *Origins of Human Communication*. Cambridge, MA, USA: MIT Press, 2008.

[15] L. Xiao and T. Yamasaki, "Probing prompt design for socially compliant robot navigation with vision-language models," 2026, accessed 2026-02-26.

[16] Z. Yan, N. Jouandeau, and A. A. Cherif, "A survey and analysis of multi-robot coordination," *International Journal of Advanced Robotic Systems*, vol. 10, no. 12, 2013, accessed 2026-02-26. [Online]. Available: https://journals.sagepub.com/doi/full/10.5772/57313

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[18] C. Zhang and V. Lesser, "Coordinating multi-agent reinforcement learning with limited communication," in *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2013, accessed 2026-02-26. [Online]. Available: https://www.ifaamas.org/Proceedings/aamas2013/docs/p1101.pdf

A Appendix

A.1 Reproducibility: Running the Controllers

The experiments in this thesis are executed from the Battery-TA-RWARE repository in this workspace. Example commands:

```
python Battery-TA-RWARE/scripts/run_llm.py \
  --env_id tarware-tiny-1agvs-1pickers-partialobs-chg-v1 \
  --seed 0 \
  --num_episodes 1 \
  --model llama3.2:3b
```

Listing 1: Run the basic per-agent language-model controller for one episode.

```
python Battery-TA-RWARE/scripts/run_centralized_llm_experiments.py \
  --prompt_format language \
  --seed 0
```

Listing 2: Run the centralized shelf-delivery experiments.

```
python Battery-TA-RWARE/scripts/run_shared_context_llm_expt.py \
  --prompt_format language \
  --seed 0
```

Listing 3: Run the shared-context shelf-delivery experiments.

```
python Battery-TA-RWARE/scripts/run_obj2_shared_context_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 4: Run the Objective 2 shared-context call-aware experiments.

```
python Battery-TA-RWARE/scripts/run_heuristic.py \
  --env_id tarware-tiny-1agvs-1pickers-partialobs-chg-v1 \
  --seed 0 \
  --num_episodes 1
```

Listing 5: Run the heuristic policy for one episode.

The main result folders discussed in the thesis are:

- results/shelf/central_llm_experiments: Objective 1, centralized planning.
- results/shelf/shared_context_llm_experiments: Objective 1, shared-context per-agent planning.
- results/calls/shared_context: Objective 2, shared-context per-agent planning with call-efficiency objective.
- results/battery: planned battery-oriented experiments (not yet populated with completed aggregate sessions at the time of writing).

A.2 Prompt Excerpt (Illustrative)

The LM is prompted with an explicit mapping from relevant state elements to discrete action identifiers. The exact prompt depends on the experiment family: centralized prompts describe all agents at once, while shared-context prompts describe one current agent together with selected global context. The full prompts are generated programmatically; the excerpt below illustrates the style (line breaks and ordering may vary):

```
Current global state:
- Requested shelves:
  - Requested shelf 24 is at position (11,5). To go there, choose
    action id 50.
  - Requested shelf 39 is at position (6,8). To go there, choose
    action id 29.
...
You are agent_0 (type=AGV).
Choose exactly one action_id from the candidate list.
Output format: action_id=<integer>
```

Listing 6: Illustrative prompt excerpt.

A.3 Metric Definitions

- **Total shelf deliveries:** number of requested shelves delivered to goal locations during an episode.
- **Overall pick rate:** deliveries/hour computed as

$$\text{pick_rate} = \frac{\text{deliveries} \cdot 3600}{5 \cdot \text{episode_steps}},$$

  where 5 steps/second is the environment's time scale used by the logging script.

- **Clashes and stuck events:** environment-defined counts related to collision resolution and deadlock-like behaviour.
- **Global episode return:** sum of environment rewards across all agents and time steps.